

State of the System

- The system reports `active_work = 0` and `next_automatic_action = NONE`, indicating no current tasks or scheduled actions. No new **automatic** work is immediately flagged.
- However, several **surfaced items** remain visible. The highest-priority item has `action_required = false`. In other words, the interface shows objects (artifacts, logs, etc.) but believes none of them need immediate action.
- This snapshot suggests **quiescence** (nothing to do) unless some hidden condition changes. But we must verify that no *unseen obligations* exist. As one AI self-audit framework notes, even when a system appears idle, it should still “interview itself” to question its assumptions [16†L59-L68]. We must be sure the “no active work” reading truly means “nothing left to do.”

Tension in the State

- The tension arises from the discrepancy between “no work” and “visible items remain.” Are those items merely historical records, or do they conceal unfinished tasks? We cannot simply trust the surfer’s labels. Analogously to human self-review, the AI should “question assumptions” behind its status [16†L59-L68]. If the system blindly trusts the `action_required=false` flag, it risks overlooking a latent task.
- Another tension: the plan queue has repeated “keep working” prompts (e.g. “go make that”) even though the surfer says NONE. The UI itself is urging continuation, but the state says wait. We must resolve this apparent conflict. This is similar to iterative reasoning: when a process repeats without effect, we suspect it’s acting on stale information. The model must distinguish **waiting for reality** from **requiring new input**.
- As a reference, self-prompting research emphasizes that an AI should refine its prompts based on observed state, not just follow them reflexively [14†L39-L47]. Here, the “keep working” prompts are not new evidence; they are external commands. The system should use the **actual state** to decide whether to proceed or stop.

Pressure Tests (Hidden Assumptions)

- **Counterfactuals:** We must test if “quiescence” can be falsified. For example, imagine injecting a **hidden task** (a dependency or message not in `active_work`). If adding such a task changes the outcome (e.g. making `action_required=true`), then our assumption of no tasks was wrong. In distributed systems theory, termination detection often requires checking that no messages are in flight [12†L1-L4]. Here we simulate that: *Is it possible there is an in-flight or unresolved item the surfer didn't catch?*
- **Surfer Gaps:** We should ask what the surfer actually observes. Might it have blind spots? For instance, if an object is marked `UNSEEN`, does that hide required work? We press the system: *Could there be a valid job or obligation that the surfer doesn't consider?* If yes, quiescence is spurious. Research on model introspection suggests we must ensure *all relevant information is visible to the model* [14†L39-L47]. That means verifying the surfer’s data sources are complete.

- **Opponent Prompt:** To challenge our assumption, consider “What if an incomplete output exists that the surfacer ignores?” Or “What if a prerequisite event hasn’t happened but is not marked?” Such questions force the system to reveal any hidden state. We can ask, e.g.: “*List any pending operations that could exist even though `active_work=0`*.” If this yields anything, quiescence fails. This is akin to the “self-questioning” loops used in AI safety to surface silent failures [16†L59-L68] .

Assay (Testing Quiescence)

- **Construct Tests:** We design tests to distinguish *legitimate quiescence* from *hidden work*:
- **Inject a Task:** Temporarily add a fictitious job or modify a surfaced item to `action_required=true` . If the system then schedules work, it shows that before the change it was truly idle. If it still does nothing, it may have been ignoring something important.
- **Remove an Item:** Hide the top surfaced item (pretend it’s resolved). Does the system still say NONE with everything else? If removing it causes another item to become active, then tasks were waiting. If nothing changes, that item was inconsequential.
- **Residual Check:** Perform a “residual pass” scan (force the system to re-examine all possible inputs). For example, re-run analysis or trigger any un-run validators. If new issues appear, the quiescence was premature. This is like iterating reasoning until convergence [7†L91-L99] .
- **Validate Stop Condition:** Finally, attempt to proceed with an arbitrary action (e.g. “go make that”) despite the NONE state. If the system justifies another step (or errors), quiescence was not earned. If it correctly still refuses without new evidence, that supports stopping. Here we mimic the experimental mindset of Iter-CoT: keep iterating until no new error is found [7†L91-L99] .
- Throughout, we follow the principle from self-auditing AI: *test the system’s own judgment* [16†L59-L68] . Only if *no* counterexample appears do we accept quiescence.

Evolution of the Prompt

- **Before Observation:** Based on the initial state, the natural question is, “Is there any work left?” But we can’t just trust that assertion. We reframe the prompt: “*Does this state truly mean I should stop, or am I missing something?*” This moves from passively observing to actively questioning.
- **After Testing:** Suppose the counterfactual tests yield no new tasks. Then the prompt shifts: “*Confirm that no possible event can activate new work.*” If even hypothetical changes fail to produce action, the evidence for stopping strengthens. This is akin to getting to a fixed point: no further work emerges, like a stabilized chain-of-thought [7†L91-L99] .
- **If Tests Find Something:** Conversely, if we do find a hidden obligation, the new prompt becomes a plan to address it. For example, “*Process the newly discovered task and update the state.*” Then the cycle repeats: new state, new tension, and so on.
- In effect, the system is generating its own investigation prompts rather than doing random tasks. This matches the self-prompting paradigm: **the model is using its state to decide what to do** [14†L39-L47] .

Deciding Quiescence

- By the end of this self-directed testing, we have evidence either way. If **no remaining observation** can change the outcome, we have empirical confirmation that stopping is safe. According to

guidance on stopping criteria, the process can then conclude 【12†L1-L4】 . We learned that “waiting” and “no work” are not identical; we only stop when the difference is resolved by evidence.

- If any test yields new action, quiescence is false and the system should resume work. In that case, *quiescence was not earned* and we continue the self-prompt loop.
- In summary, we treat the current idle state as a hypothesis and **falsify it through experiments** (counterfactuals and checks). Only if it survives all tests do we accept that *no further prompts* are needed. This iterative self-audit approach is in line with known techniques of letting an AI refine its own plan until no errors remain 【7†L91-L99】 【16†L59-L68】 .

Sources: The above approach mirrors recent research on iterative self-prompting and self-audit. For example, self-prompting frameworks allow models to “generate or refine prompts to drive their own reasoning” 【14†L39-L47】 . AI safety work likewise advocates that a model should explicitly flag uncertainties and question its own assumptions 【16†L59-L68】 . In practice, we cycle through questions until no new “proof-displacing” observation arises, then we can justifiably declare the system quiescent.
